



Jigsaw Audit Report

Version 1.0

Oxenzo

May 21, 2025

Prepared by: 0xenzo

Lead Security Researcher: 0xenzo

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues Found
- Findings
 - Medium
 - * [M-1] `getHoldingFee` function may return 0 instead of default performance fee.
 - * [M-2] While `HoldingManager` contract is paused, positions could go liquidatable.
 - Low
 - * [L-1] Incorrect Upper bound check in `setPerformanceFee` function.
 - * [L-2] Incorrect upper bound check for the `_setHoldingCustomFee` function.
 - Informational
 - * [I-1] Misleading comments for the `isTokenWithdrawable` mapping that is later used in various functions.
 - * [I-2] Missing 50bps fee in `borrow()` function leads to revenue loss.

Protocol Summary

Jigsaw is a CDP-based stablecoin protocol that brings full flexibility and composability to your collateral through the concept of “dynamic collateral”. Jigsaw leverages crypto’s unique permissionless composability to enable dynamic collateral in a fully non-custodial way.

Disclaimer

I, 0xzenzo makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by me is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

I have used the Cantina severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond to the following commit hash:

```
1 e8f373e0f5c7416bd0acee800223e6fd6a959494
```

Scope

The following contracts are in scope for this audit:

Contract
src/Holding.sol
src/HoldingManager.sol
src/JigsawUSD.sol
src/LiquidationManager.sol
src/Manager.sol
src/ReceiptToken.sol
src/ReceiptTokenFactory.sol
src/SharesRegistry.sol
src/StablesManager.sol
src/StrategyManager.sol
src/SwapManager.sol

Roles

Roles can be seen from the codebase.

Executive Summary

This audit was conducted on the Jigsaw Finance protocol, covering the core contracts responsible for holdings management, liquidation, fee handling, and stablecoin minting.

Over the course of the audit, I identified **2 medium severity**, **2 low severity**, and **2 informational** findings.

The medium severity findings centred around fee misconfiguration and borrower protection during emergency pauses. Specifically, the `getHoldingFee` function incorrectly falls back to a zero fee instead of the protocol's default performance fee, creating unintended fee waivers that reduce protocol revenue. Additionally, the `HoldingManager` pause mechanism blocks borrowers from repaying their debt, leaving positions exposed to liquidation through no fault of the user once the protocol resumes.

The low severity findings both relate to off-by-one boundary checks in fee setter functions across `Manager.sol` and `FeeManager.sol`, where strict less-than comparisons incorrectly prevent setting fees to the documented maximum allowed value.

On the informational side, I flagged a misleading comment on the `isTokenWithdrawable` mapping that inverts its actual semantics, as well as a missing 50bps minting fee in the `borrow` function that contradicts the protocol's documented fee structure and results in a direct loss of expected revenue.

Overall, the codebase is reasonably structured, but the findings above — particularly around fee logic and pause mechanics — warrant attention before deployment.

Issues Found

Severity	Number of Issues Found
High	0
Medium	2
Low	2
Info	2
Total	6

Findings

Medium

[M-1] `getHoldingFee` function may return 0 instead of default performance fee.

Description:

The `getHoldingFee` function aims to return a custom fee if set, otherwise default to the strategy's `performance fee`:

<https://github.com/jigsaw-finance/jigsaw-strategies-v1/blob/aa477065672643777143269615818b4cdd27c02f/src/ex>

However, this function will still return the default performance fee as zero because its basically checking for the strategy's performance fee instead of the default performance fee stated in the `Manager.sol` file:

<https://github.com/jigsaw-finance/jigsaw-protocol-v1/blob/209bceaa5ed4b837ef065c80a3467740a6bc683/src/Manager.sol#L99C1>

thus contradicting the documented expectation.

Impact:

A missing fee on `holdings` without custom fees can lead to unintended fee waivers, reducing protocol revenue.

Proof of Concept:

Would require to place this in the `FeeManager.t.sol` file:

```
1 function test_returnsDefaultFeeWhenCustomFeeNotSet() public {
2     (uint256 defaultPerformanceFee,,) = IStrategyManager(manager.
3         strategyManager()).strategyInfo(address(strategy));
4     uint256 fee = feeManager.getHoldingFee(holding, strategy);
5     assertEq(fee, defaultPerformanceFee, "Should return default fee
6         when custom fee is not set");
7 }
```

This test will pass unfortunately with the `defaultPerformanceFee` being zero as seen from the logs:

```

1  |
2  [768] StrategyManager::strategyInfo(RIPEMD-160: [0
      x0000000000000000000000000000000000000000000000000000003]) [staticcall] || ↵
3      [Return] 0, false, false | ↵
4      [Return] 0 |
5      [0] VM::assertEq(0, 0, "Should return default fee when custom fee
      is not set") [staticcall] | ↵
6      [Return] ↵
7      [Stop]

```

Recommended Mitigation:

In this branch of the function -> <https://github.com/jigsaw-finance/jigsaw-strategies-v1/blob/aa477065672643777143>

I would require to return the default performance Fee from the `Manager.sol` file which can be found here:

<https://github.com/jigsaw-finance/jigsaw-protocol-v1/blob/209bceaa5ed4b837ef065c80a3467740a6bc683/src/MarL99C1>

And also I would require a zero fee validation in the form of this, before returning the `defaultPerformanceFee` from the function.

```
1 require(defaultPerformanceFee > 0, "No performance fee set in manager")
    ;
```

[M-2] While HoldingManager contract is paused, positions could go liquidatable.

Description:

The `HoldingManager` contract makes use of the `whenNotPaused` modifier (from the `Pausable` contract) to prevent the execution of certain functions when the protocol is paused. One such function is `repay` which is essential for borrowers to manage their debt and solvency.

The `repay` function with `whenNotPaused` modifier -> <https://github.com/jigsaw-finance/jigsaw-protocol-v1/blob/209bceea5ed4b837ef065c80a3467740a6bc683/src/HoldingManager.sol#L358>

while the intention behind pausing is to halt protocol activity during an emergency, this design introduces an unintended side effect: `borrowers are blocked from repaying their loans`, which could cause their positions to fall below the liquidation threshold due to external market movements.

Once the protocol is unpaused, these borrowers become immediately `liquidatable`, creating an unfair race and potentially causing them to lose their collateral due to circumstances beyond their control.

Impact:

`Borrowers` cannot proactively manage their positions while paused.

Proof of Concept:

Kindly place this in the `Liquidation.t.sol` file.

```
1
2 function test_positiongetsLiquidated_DuringPausePeriod() public {
3     uint256 _collateralAmount = 10_000e22;
4
5     TestTempData memory testData;
6
7     // initialize user
8     testData.user = user;
9     testData.userCollateralAmount = _collateralAmount;
10    SampleTokenBigDecimals collateralContract = new
        SampleTokenBigDecimals("BigDec", "BD", 1e18 * 1e22);
11    manager.whitelistToken(address(collateralContract));
12    SampleOracle collateralOracle = new SampleOracle();
13    SharesRegistry collateralRegistry = new SharesRegistry(
14        msg.sender,
15        address(manager),
16        address(collateralContract),
17        address(collateralOracle),
18        bytes(""),
19        ISharesRegistry.RegistryConfig({
20            collateralizationRate: 50_000,
21            liquidationBuffer: 5e3,
22            liquidatorBonus: 8e3
23        })
24    );
25    registries[address(collateralContract)] = address(
        collateralRegistry);
26    stablesManager.registerOrUpdateShareRegistry(address(
        collateralRegistry), address(collateralContract), true);
27
```

```

28     // calculate mintAmount
29     uint256 mintAmount = testData.userCollateralAmount / 2;
30
31     //get tokens for user
32     deal(address(collateralContract), testData.user, testData.
        userCollateralAmount);
33
34     vm.startPrank(testData.user, testData.user);
35     // create holding for user
36     testData.userHolding = holdingManager.createHolding();
37     // make deposit to the holding
38     collateralContract.approve(address(holdingManager), testData.
        userCollateralAmount);
39     holdingManager.deposit(address(collateralContract), testData.
        userCollateralAmount);
40     //borrow
41     holdingManager.borrow(address(collateralContract), mintAmount,
        0, true); // 5000e22
42     vm.stopPrank();
43
44     testData.userJUsd = jUsd.balanceOf(testData.user); // 5000e22
45
46     //initialize liquidator
47     testData.liquidator = liquidator;
48     testData.liquidatorCollateralAmount = _collateralAmount; //
        10000e22
49
50     //get tokens for user
51     deal(address(collateralContract), testData.liquidator, testData.
        liquidatorCollateralAmount);
52
53     vm.startPrank(testData.liquidator, testData.liquidator);
54     // create holding for user
55     holdingManager.createHolding();
56     // make deposit to the holding
57     collateralContract.approve(address(holdingManager), testData.
        userCollateralAmount);
58     holdingManager.deposit(address(collateralContract), testData.
        userCollateralAmount);
59     //borrow
60     holdingManager.borrow(address(collateralContract), mintAmount,
        0, true); // 5000e22
61     vm.stopPrank();
62
63     testData.liquidatorJUsd = jUsd.balanceOf(testData.liquidator);
        // 5000e22
64     testData.liquidatorCollateralAmountAfterInitiation =
        collateralContract.balanceOf(address(testData.liquidator));
        // 0
65
66     //change the price of the collateral

```

```
67     collateralOracle.setPriceForLiquidation(); // 8e17 below
        borrowed amount of 5000e22
68
69     // OverHere the HoldingManager pauses the contract.
70     vm.startPrank(address(this));
71     holdingManager.pause();
72     vm.stopPrank();
73
74     // User with liquidatable position tries to repay to get back
        most collateral over here.
75     // the repay method reverts due to the function being paused.
76     vm.startPrank(testData.user);
77     vm.expectRevert();
78     holdingManager.repay(address(collateralContract), mintAmount,
        true);
79     vm.stopPrank();
80
81     // Liquidator swoops in to liquidate the user.
82
83     //initiate liquidation from liquidator's address
84     vm.startPrank(testData.liquidator, testData.liquidator);
85
86     ILiquidationManager.LiquidateCalldata memory liquidateCalldata;
87
88     //make liquidation call
89     liquidationManager.liquidate(
90         address(testData.user), address(collateralContract),
            testData.userJUsd, 0, liquidateCalldata
91     );
92
93     vm.stopPrank();
94 }
```

Recommended Mitigation:

Implement a `post-unpause grace period` before liquidations can be executed.

Low

[L-1] Incorrect Upper bound check in `setPerformanceFee` function.

Description:

The `setPerformanceFee` function is intended to update the `performanceFee` variable, ensuring that the new value does not exceed the maximum allowed performance fee defined by `MAX_PERFORMANCE_FEE`. However, the current implementation uses a strict less-than `<` comparison.

[https://github.com/jigsaw-finance/jigsaw-protocol-v1/blob/209bceea5ed4b837ef065c80a3467740a6bc683/src/Manager](https://github.com/jigsaw-finance/jigsaw-protocol-v1/blob/209bceea5ed4b837ef065c80a3467740a6bc683/src/Manager.sol)

This condition erroneously disallows setting the performance fee to exactly `MAX_PERFORMANCE_FEE`, which contradicts the documented behavior. The documentation specifies that `MAX_PERFORMANCE_FEE` represents the maximum allowable fee, implying that setting the fee to this exact value should be permissible.

Impact:

Inconsistency: Creates a discrepancy between the contract's documentation and its actual behavior, which can lead to confusion and misconfiguration.

Proof of Concept:

1. Assume `MAX_PERFORMANCE_FEE` is set to 2500 (representing 25 %).
2. Attempting to call `setPerformanceFee(2500)` will revert due to the strict less-than check, despite 2500 being the documented max allowable value.

Place this in the `Manager.sol` file to check it out:

```
1 function test_setMax_performance_fee() public {
2     uint256 maxFee = manager.MAX_PERFORMANCE_FEE(); // which is
        2500; //25%
3
4     vm.startPrank(OWNER);
5     vm.expectRevert(bytes("3018"));
6     manager.setPerformanceFee(maxFee);
7     vm.stopPrank();
8 }
```

Recommended Mitigation:

Modify the `require` statement to use a less-than-or-equal-to (`<=`) comparison to align with the documented behavior:

```
1 function setPerformanceFee(
2     uint256 _val
3 ) external override onlyOwner {
4     require(performanceFee != _val, "3017");
5 -     require(_val < MAX_PERFORMANCE_FEE, "3018");
6 +     require(_val <= MAX_PERFORMANCE_FEE, "3018");
7     emit PerformanceFeeUpdated(performanceFee, _val);
8     performanceFee = _val;
9 }
```

[L-2] Incorrect upper bound check for the `_setHoldingCustomFee` function.

Description:

The `_setHoldingCustomFee` function in `FeeManager.sol` performs a strict “less than” comparison against the protocol’s maximum performance fee from `Manager.sol` as this can be seen from the link below:

<https://github.com/jigsaw-finance/jigsaw-strategies-v1/blob/aa477065672643777143269615818b4cdd27c02f/src/ex>

But when the Max Performance Fee which is stated here is put:

<https://github.com/jigsaw-finance/jigsaw-protocol-v1/blob/209bceea5ed4b837ef065c80a3467740a6bc683/src/Man>

However, Jigsaw’s documentation defines `MAX_PERFORMANCE_FEE` as the upper limit, meaning setting a fee equal to this value should be allowed—but the current check prohibits it. This inconsistency prevents users from configuring the fee at the maximum intended level.

Impact:

Results in a configuration mismatch between protocol expectations and actual behavior.

Proof of Concept:

Would require to put this in the `FeeManager.t.sol` file.

```
1 function test_revertOnSettingDefaultMaxPerformanceFee() public {
2     uint256 maxPerformanceFee = manager.MAX_PERFORMANCE_FEE();
3
4     vm.startPrank(OWNER);
5     vm.expectRevert(bytes("3018"));
6     feeManager.setHoldingCustomFee(holding, strategy,
7         maxPerformanceFee);
8     vm.stopPrank();
9 }
```

Despite setting `_fee` to the exact `MAX_PERFORMANCE_FEE`, the function incorrectly reverts.

Recommended Mitigation:

Update the comparison to allow the maximum value.

```
1 function _setHoldingCustomFee(address _holding, address _strategy,
2     uint256 _fee) private {
3     require(_strategy != address(0), "3000");
4     require(_holding != address(0), "3000");
5     - require(_fee < manager.MAX_PERFORMANCE_FEE(), "3018");
6     + require(_fee <= manager.MAX_PERFORMANCE_FEE(), "3018");
7     require(holdingFee[_holding][_strategy] != _fee, "3017");
8
9     emit HoldingFeeUpdated({
10         holding: _holding,
11         strategy: _strategy,
12         oldFee: holdingFee[_holding][_strategy],
13         newFee: _fee
14     });
15 }
```

```
13     });
14     holdingFee[_holding][_strategy] = _fee;
15 }
```

Informational

[I-1] Misleading comments for the `isTokenWithdrawable` mapping that is later used in various functions.

Description:

The `isTokenWithdrawable` mapping has comments stating that `Returns true if the token cannot be withdrawn from a holding.`

<https://github.com/jigsaw-finance/jigsaw-protocol-v1/blob/94636c0bd30f1c96327300654e283ce2931df84d/src/Manager.sol#L13>

But when used in other functions that's not the case as we can see in `addWithdrawableToken`, `removeWithdrawableToken`.

<https://github.com/jigsaw-finance/jigsaw-protocol-v1/blob/94636c0bd30f1c96327300654e283ce2931df84d/src/Manager.sol#L13>

<https://github.com/jigsaw-finance/jigsaw-protocol-v1/blob/94636c0bd30f1c96327300654e283ce2931df84d/src/Manager.sol#L13>

This strongly implies that `true` = withdrawable, not the opposite.

Impact:

Critical authorization logic or withdrawal flows may be misinterpreted and mistakenly altered or overlooked.

Proof of Concept:

For instance in the `Manager::addWithdrawableToken`, this line is present.

```
1     isTokenWithdrawable[token] = true;
```

Meaning that `true` correlates with withdrawable.

Recommended Mitigation:

Just update the comment about the mapping.

```
1  /**
2   * @notice Returns true if the token cannot be withdrawn from a
3   *         holding.
4   */
5  /**
6   * @notice Returns true if the token can be withdrawn from a holding.
7   */
```

```
7
8 mapping(address token => bool withdrawable) public override
    isTokenWithdrawable;
```

[I-2] Missing 50bps fee in borrow() function leads to revenue loss.

Description:

The protocol's documentation specifies a 50 basis points (0.5%) fee on minting JUSD to generate protocol revenue.

<https://jigsaw.gitbook.io/jigsaw-protocol/tokenomics/fee-structure>

However the `borrow` function in `StablesManager.sol` does not implement this fee, allowing users to mint JUSD without incurring the intended charge.

Impact:

The omission leads to a direct loss of expected revenue for the protocol and may result in inconsistencies between documented behaviour and actual implementation.

Proof of Concept:

In the `borrow` function, the amount of jUSD minted is calculated without applying the 0.5% fee.

<https://github.com/jigsaw-finance/jigsaw-protocol-v1/blob/209bceaa5ed4b837ef065c80a3467740a6bc683/src/StablesManager.sol#L100>

This calculation does not deduct the 50bps fee, resulting in users receiving the full amount of jUSD without any fee applied.

From the POC below;

CR = 50% has been implemented by the way.

Place this in the `StablesManager.t.sol`:

add this storage variable first;

```
1 address USER1 = makeAddr("USER1");
```

Then,

```
1 function test_borrow_with_no_MintingFee_for_JUSD() public {
2
3     // get USDC for user.
4     deal(address(usdc), USER1, 1000e18);
5
6     // Whitelist the contract by Manager so as to create a holding
7     vm.startPrank(OWNER);
8     manager.whitelistContract(USER1);
9 }
```

```
9      vm.stopPrank();
10
11      // USER creates its Own Holding.
12      vm.startPrank(USER1);
13      address holdingAddress = holdingManager.createHolding();
14      vm.stopPrank();
15
16      // Now Deposit Collateral to holding through holdingManager
17      // since we using a EOA..
18      vm.startPrank(USER1);
19      usdc.approve(address(holdingManager), type(uint256).max);
20      holdingManager.deposit(address(usdc), 500e18);
21      vm.stopPrank();
22
23      // Let's say now USER wants to borrow.
24      vm.startPrank(USER1);
25      holdingManager.borrow(address(usdc), 250e18, 0, true);
26      vm.stopPrank();
27
28      // assert the amount of JUSD user has assuming CR was
29      // considered in the borrow function.
30      // no mint fee has been enforced by the way.
31      assertEq(jUsd.balanceOf(USER1), 250e18);
32  }
```

This test above is going to pass, since the user will get the full **JUSD** amount.

Recommended Mitigation:

Incorporate the 50bps fee into the **borrow** function to align with the protocols documentation and ensure revenue generation.